

추상 클래스(**abstract class**)는 인터페이스의 역할도 하면서 클래스의 기능도 가지고 있는 특별한 클래스이다. 추상 클래스는 인터페이스처럼 추상 메서드를 가질 수 있으면서도, 일반 클래스처럼 구현된 메서드, 생성자, 인스턴스 변수 등을 가질 수 있다. 이러한 추상 클래스를 알아보기 위해 우리가 작성했던 **Predator** 인터페이스를 다음과 같이 추상 클래스로 변경해 보자.

앞서 언급했듯이 05장에서 사용되는 예제는 모두 연속되므로 순서대로 예제를 따라 해야 한다.

```
abstract class Predator extends Animal {  
    abstract String getFood();  
  
    default void printFood() { // default 를 제거한다.  
        System.out.printf("my food is %s\n", getFood());  
    }  
}  
  
(... 생략 ...)
```

추상 클래스를 만들려면 **class** 앞에 **abstract** 키워드를 붙여야 한다. 또한 인터페이스의 메서드와 같은 역할을 하는 메서드(여기서는 **getFood** 메서드)에도 **abstract**를 붙여야 한다. **abstract** 메서드는 인터페이스의 메서드와 마찬가지로 메서드 본문이 없는 선언만 존재한다. 따라서 추상 클래스를 상속하는 자식 클래스에서는 반드시 모든 **abstract** 메서드를 구현해야 한다. 그리고 **Animal** 클래스의 기능을 유지하기 위해 **Animal** 클래스를 상속했다.

추상 클래스에서는 인터페이스의 **default** 메서드 문법을 사용할 수 없으므로 **default** 키워드를 삭제하여 일반 메서드로 변경했다.

추상 클래스는 일반 클래스와 달리 단독으로 객체를 생성할 수 없다. 반드시 추상 클래스를 상속한 실제 클래스를 통해서만 객체를 생성할 수 있다.

Predator 인터페이스를 이와 같이 추상 클래스로 변경하면 Predator 인터페이스를 상속했던 BarkablePredator 인터페이스는 더 이상 사용이 불가능하므로, 다음과 같이 삭제해야 한다. 그리고 Tiger, Lion 클래스도 Animal 클래스 대신 Predator 추상 클래스를 상속하도록 변경해야 한다.

```
abstract class Predator extends Animal {
    (... 생략 ...)
}

interface Barkable {
    (... 생략 ...)
}

interface BarkablePredator extends Predator, Barkable {
}

class Animal {
    (... 생략 ...)
}

class Tiger extends Predator implements Barkable {
    (... 생략 ...)
}

class Lion extends Predator implements Barkable {
    (... 생략 ...)
}

class ZooKeeper {
    (... 생략 ...)
}

class Bouncer {
    (... 생략 ...)
}

public class Sample {
    (... 생략 ...)
}
```

Predator 추상 클래스에 선언된 getFood 메서드는 Tiger, Lion 클래스에 이미 구현되어 있으므로 추가로 구현할 필요는 없다. 추상 클래스의 abstract 메서드는 인터페이스의 메서드와 마찬가지로 상속받는 클래스에서 반드시 구현해야 한다. 추상 클래스의 큰 장점은 abstract 메서드뿐만 아니라 일반 메서드(구현체가 있는 메서드)도 함께 가질 수 있다는 점이다. 추상 클래스에 일반 메서드를 추가하면 Tiger, Lion 등 상속받은 모든 객체에서 그 메서드를 바로 사용할 수 있다. 원래 인터페이스에서 default 메서드로 사용했던 printFood가 추상 클래스의 일반 메서드에 해당된다.

추상 클래스의 특징